Group Project - Modeling and Variable Selection

Gabriel Arsego

2026-05-01

Loading the datasets:

```
library(data.table)
library(caret)
```

```
## Loading required package: ggplot2

## Loading required package: lattice
```

```
library(glmnet)
```

```
## Loading required package: Matrix

## Loaded glmnet 4.1-10
```

```
set.seed(123)

setwd("C:/Users/arseg/Desktop")

train_clean <- fread("train_clean_final.csv")
test_clean <- fread("test_clean_final.csv")

train_model <- as.data.frame(train_clean)
test_model <- as.data.frame(test_clean)
```

Final confirmation before modeling:

```
sum(is.na(train_model))
```

```
## [1] 0
```

```
sum(is.na(test_model))
```

```
## [1] 0
```

```
names(train_model)[!sapply(train_model, is.numeric)]
```

```
## [1] "f531"
```

```r
names(test_model)[!sapply(test_model, is.numeric)]
```

```
## character(0)
```

```r
"loss" %in% names(train_model)
```

```
## [1] TRUE
```

```r
train_model$f531 <- NULL
```

Creating a training and a validation set:

```r
index <- createDataPartition(train_model$loss, p = 0.7, list = FALSE)

training <- train_model[index, ]
validation <- train_model[-index, ]

dim(training)
```

```
## [1] 56000   731
```

```r
dim(validation)
```

```
## [1] 24000   731
```

Here I'm creating a baseline Linear Regression with the top 50 variables in correlation with "loss":

```r
# Identify predictor columns
predictor_cols <- setdiff(names(training), "loss")

# Calculate correlation between each predictor and loss
correlations <- sapply(training[, predictor_cols], function(x) {
  cor(x, training$loss, use = "complete.obs")
})

# Select top 50 predictors by absolute correlation
top_features <- names(sort(abs(correlations), decreasing = TRUE))[1:50]

top_features
```

```
##  [1] "f471" "f536" "f468" "f617" "f674" "f67"  "f68"  "f533" "f599" "f413"
## [11] "f13"  "f251" "f620" "f221" "f314" "f315" "f323" "f281" "f322" "f526"
## [21] "f70"  "f675" "f598" "f282" "f596" "f621" "f252" "f676" "f765" "f405"
## [31] "f767" "f615" "f400" "f69"  "f734" "f775" "f290" "f200" "f111" "f738"
## [41] "f299" "f556" "f776" "f80"  "f666" "f404" "f665" "f755" "f428" "f766"
```

Creating smaller training and validation datasets (so I can actually run this):

```r
training_small <- training[, c("loss", top_features)]

validation_small <- validation[, c("loss", top_features)]
```

Running the Linear Regression:

```r
lm_model <- lm(loss ~ ., data = training_small)

lm_pred <- predict(lm_model, newdata = validation_small)

lm_results <- postResample(pred = lm_pred, obs = validation_small$loss)

lm_results
```

```
##        RMSE    Rsquared         MAE
## 4.152607670 0.004569884 1.431665514
```

Now I'll run Lasso Regression as Kight Pruszynski did:

```r
x_train <- as.matrix(training[, setdiff(names(training), "loss")])
y_train <- training$loss

x_val <- as.matrix(validation[, setdiff(names(validation), "loss")])
y_val <- validation$loss
```

Running Lasso with cross-validation:

```r
set.seed(123)

lasso_cv <- cv.glmnet(
  x = x_train,
  y = y_train,
  alpha = 1,
  nfolds = 5
)
```

Checking the best lambda:

```r
lasso_cv$lambda.min
```

```
## [1] 0.02875202
```

```r
lasso_cv$lambda.1se
```

```
## [1] 0.1848201
```

Predicting on validation data:

```r
lasso_pred <- predict(lasso_cv, s = "lambda.min", newx = x_val)

lasso_results <- postResample(
  pred = as.numeric(lasso_pred),
  obs = y_val
)

lasso_results
```

```
##        RMSE    Rsquared         MAE
## 4.154151957 0.004359955 1.430741350
```

Running Ridge Regression with cross-validation:

```r
set.seed(123)

ridge_cv <- cv.glmnet(
  x = x_train,
  y = y_train,
  alpha = 0,
  nfolds = 5
)
```

Checking the best lambda for Ridge:

```r
ridge_cv$lambda.min
```

```
## [1] 5.387604
```

```r
ridge_cv$lambda.1se
```

```
## [1] 184.8201
```

Predicting on validation data using Ridge:

```r
ridge_pred <- predict(ridge_cv, s = "lambda.min", newx = x_val)

ridge_results <- postResample(
  pred = as.numeric(ridge_pred),
  obs = y_val
)

ridge_results
```

```
##        RMSE    Rsquared         MAE
## 4.155861300 0.003093052 1.433639100
```

Running Elastic Net Regression with cross-validation:

```r
set.seed(123)

enet_cv <- cv.glmnet(
  x = x_train,
  y = y_train,
  alpha = 0.5,
  nfolds = 5
)
```

Checking the best lambda for Elastic Net:

```r
enet_cv$lambda.min
```

```
## [1] 0.05750404
```

```r
enet_cv$lambda.1se
```

```
## [1] 0.3696402
```

Predicting on validation data using Elastic Net:

```r
enet_pred <- predict(enet_cv, s = "lambda.min", newx = x_val)

enet_results <- postResample(
  pred = as.numeric(enet_pred),
  obs = y_val
)

enet_results
```

```
##        RMSE    Rsquared         MAE
## 4.154230858 0.004334956 1.430822021
```

Comparing all models:

```r
model_results <- rbind(
  Linear_Regression_Top50 = lm_results,
  Lasso_All_Variables = lasso_results,
  Ridge_All_Variables = ridge_results,
  Elastic_Net_All_Variables = enet_results
)

model_results[order(model_results[, "MAE"]), ]
```

```
##                               RMSE    Rsquared      MAE
## Lasso_All_Variables       4.154152 0.004359955 1.430741
## Elastic_Net_All_Variables 4.154231 0.004334956 1.430822
## Linear_Regression_Top50   4.152608 0.004569884 1.431666
## Ridge_All_Variables       4.155861 0.003093052 1.433639
```

Lasso has the lowest MAE. Checking which variables Lasso selected:

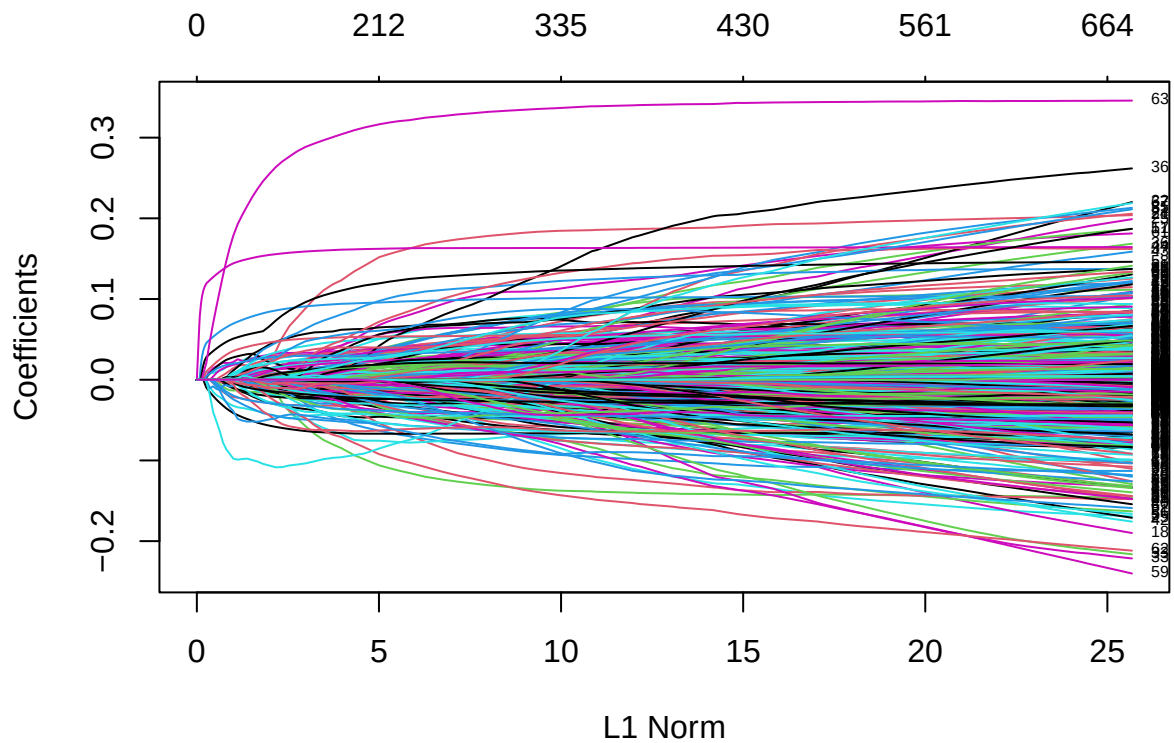
```r
lasso_coef <- coef(lasso_cv, s = "lambda.min")

lasso_coef_df <- data.frame(
  variable = rownames(lasso_coef),
  coefficient = as.numeric(lasso_coef)
)

selected_lasso_vars <- lasso_coef_df[lasso_coef_df$coefficient != 0, ]

selected_lasso_vars <- selected_lasso_vars[selected_lasso_vars$variable != "(Intercept)", ]

selected_lasso_vars
```

```
##     variable   coefficient
## 61       f67  0.0158083399
## 62       f68  0.0208144609
## 64       f70 -0.0178700683
## 74       f80  0.0156730471
## 105     f111 -0.0064901572
## 152     f160 -0.0027177216
## 192     f200 -0.0111003713
## 208     f218  0.0013268968
## 211     f221  0.0002548313
## 240     f251  0.0245985072
## 266     f281  0.0082200898
## 299     f314 -0.0272517421
## 308     f323 -0.0039433672
## 345     f361  0.0025722828
## 368     f384  0.0002136371
## 395     f413 -0.0354703485
## 447     f471  0.1395341999
## 499     f526 -0.0175778550
## 504     f533  0.0309845205
## 506     f536  0.0649451307
## 570     f604 -0.0048682641
## 582     f617  0.0468754796
## 585     f620  0.0182539789
## 628     f665 -0.0065646520
## 629     f666 -0.0862722196
## 636     f674  0.1468969825
## 641     f680  0.0023900357
## 643     f682  0.0034293352
## 702     f747 -0.0007455328
## 710     f755 -0.0282994149
## 723     f769  0.0015516922
## 729     f775 -0.0146753018
```

```r
nrow(selected_lasso_vars)
```

```
## [1] 32
```

```r
plot(lasso_cv$glmnet.fit, xvar = "norm", label = TRUE)
```



Predicting on the Cleaned Test Dataset:

```r
predictor_names <- setdiff(names(training), "loss")

test_model_final <- test_model[, predictor_names, drop = FALSE]

x_test <- as.matrix(test_model_final)

test_pred <- predict(lasso_cv, s = "lambda.min", newx = x_test)

test_pred <- as.numeric(test_pred)

test_pred[test_pred < 0] <- 0
```

Number of predictions:

```r
length(test_pred)
```

```
## [1] 25471
```

```r
nrow(test_model_final)
```

```
## [1] 25471
```

```r
test_ids <- fread("C:/Users/arseg/Desktop/test_ids.csv")
```

```r
names(test_ids)
```

```
## [1] "id"
```

```r
head(test_ids)
```

```
##        id
##     <int>
## 1:   7933
## 2: 101860
## 3:  62580
## 4:   1760
## 5:  48008
## 6:   9308
```

```r
test_id <- test_ids$id
```

```r
length(test_id)
```

```
## [1] 25471
```

```r
length(test_pred)
```

```
## [1] 25471
```

```r
nrow(test_model_final)
```

```
## [1] 25471
```

```r
group_project_loss_prediction <- data.frame(
  id = test_id,
  loss = test_pred
)
head(group_project_loss_prediction)
```

```
##       id      loss
## 1   7933 0.9731800
## 2 101860 0.8139247
## 3  62580 0.6888704
## 4   1760 0.7512310
## 5  48008 0.8476819
## 6   9308 0.7749406
```

```r
names(group_project_loss_prediction)
```

```
## [1] "id"   "loss"
```

```r
dim(group_project_loss_prediction)
```

```
## [1] 25471     2
```

```r
sum(is.na(group_project_loss_prediction))
```

```
## [1] 0
```

```r
write.csv(
  group_project_loss_prediction,
  "Group Project - Loss Prediction.csv",
  row.names = FALSE
)
```